

Informe Temático Unidad 3

Herramientas Computacionales

Estudiante: Felipe Colli Olea
Profesor: Nelson Baloian
Auxiliares: Matías Godoy
Pablo Rivas

Fecha de realización: 15 de junio de 2026
Santiago de Chile

Índice de Contenidos

1. Introducción	1
2. Trabajos Realizados	1
2.1. Evidencia de los laboratorios	2
3. Principales Dificultades	3
4. Conclusiones	3

Índice de Figuras

1. Evidencia del Laboratorio 09: Implementación de regresión lineal y procesamiento de listas dinámicas.	2
2. Evidencia del Laboratorio 10: Ejecución del sistema de votaciones y motor lógico del juego de Cachipún.	2

1. Introducción

Durante la tercera unidad del curso, el enfoque de aprendizaje transicionó desde el paradigma de las planillas de cálculo hacia la programación estructurada mediante el lenguaje **Python**. El objetivo fue desarrollar el pensamiento algorítmico, comprendiendo cómo las máquinas procesan instrucciones secuenciales, controlan flujos de ejecución (mediante ciclos y condicionales) y manipulan estructuras de datos en memoria para resolver problemas de ingeniería.

A nivel personal, poseo un historial formativo orientado a la *Programación Competitiva* utilizando lenguajes compilados de bajo nivel y tipado estático (específicamente C++). Sin embargo, esta unidad fue útil para interiorizar el paradigma de los lenguajes interpretados de alto nivel. Lo que aprendí en esta unidad, y que supuso un contraste directo con mi formación previa, fue el manejo de la vectorización implícita (*broadcasting* en librerías numéricas) y la adaptación a un lenguaje de tipado dinámico donde la abstracción de la memoria es gestionada íntegramente por el intérprete.

2. Trabajos Realizados

La unidad estuvo compuesta por cuatro laboratorios prácticos orientados a dominar distintas estructuras de control y análisis de datos:

- **Laboratorio 7 (Vectores y Método de Montecarlo):** El objetivo fue familiarizarse con la sintaxis básica y el entorno *Google Colab*. Se implementó el método estocástico de Montecarlo utilizando arreglos de generación pseudoaleatoria (**numpy.random**) para aproximar el valor de la constante matemática π , analizando la tasa de error estadístico.
- **Laboratorio 8 (Funciones y Lógica Condicional):** Orientado al control de flujo estático. Se diseñaron funciones evaluadoras de estados lógicos mediante compuertas **if/elif/else**, resolviendo problemas de clasificación numérica, evaluación de años bisieños (anidamiento lógico) y validación de la desigualdad triangular.
- **Laboratorio 9 (Listas e Iteradores Definidos):** El objetivo fue manipular colecciones dinámicas de datos utilizando ciclos **for**. Se implementó de manera manual un modelo de regresión lineal (Mínimos Cuadrados Ordinarios) iterando sobre listas crudas, y un algoritmo para procesar y modificar masivamente un arreglo de calificaciones académicas.
- **Laboratorio 10 (Ciclos Indefinidos y Máquinas de Estado):** El enfoque fue dominar la instrucción **while**. Se programó un sistema de escrutinio de votos con terminación por valor centinela (0) y un motor de juego (Cachipún), el cual fue estructurado analíticamente mediante aritmética modular en lugar de ramificaciones de fuerza bruta.

2.1. Evidencia de los laboratorios

A continuación, se adjunta evidencia visual del trabajo desarrollado en los entornos requeridos.

```

x=[10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 100]
y=[150, 160, 180, 190, 210, 220, 240, 250, 270, 280, 300, 310, 330, 340, 360, 370, 390, 400, 420]

x = []
for i in range(10):
    x.append(x * 0.0254)

y = []
for i in range(10):
    y.append(y * 0.45)

a = 0
b = 0
m = len(x)

for i in range(m):
    a += x[i]
    b += y[i]
    m += x[i] * y[i]

d = (m * a) - (a * m)
e = (m * b) - (b * m) / d
f = (e * a) - (a * m) / d

y2 = []
for i in range(m):
    y2.append((x[i] * e) + f)

print("Y | Y | Y2")
for i in range(m):
    print(x[i], y[i], y2[i])

print("\n")

# ... Función 3: Notas ...
notas = []
print("Ingresar las 15 notas:")
for i in range(15):
    val = float(input("Nota " + str(i+1) + ": "))
    notas.append(val)

notas2 = []
for i in range(15):
    if i % 2 == 0:
        notas2.append(notas[i])
    else:
        notas2.append(notas[i] * 1.25)

p = 0
p2 = 0
print("Notas Originales | Notas Mejoradas")
for i in range(15):
    print(notas[i], " ", notas2[i])
    p += notas[i]
    p2 += notas2[i]

print("\n")
print("Promedio Notas Originales:")
print(p / 15)
print("Promedio Notas Mejoradas:")
print(p2 / 15)

```

Figura 1: Evidencia del Laboratorio 09: Implementación de regresión lineal y procesamiento de listas dinámicas.

```

# ... Función 4: Votos ...
votos = []
for i in range(10):
    val = float(input("Voto " + str(i+1) + ": "))
    votos.append(val)

# ... Función 5: Resultados ...
print("Resultados:")
for i in range(10):
    print(votos[i])

# ... Función 6: Juego de Cachipún ...
def juego_cachipun():
    print("Juego de Cachipún (Al mejor de 3)...")
    print("Opciones: 1 (Papel), 2 (Jesús), 3 (Piedra)")
    c = 0
    u = 0
    while u < 3 and c < 3:
        print("Ingrese su jugada (1, 2 o 3): ")
        u = int(input())
        if u not in [1, 2, 3]:
            print("Jugada inválida, intenta de nuevo.")
            continue
        computador = random.randint(1, 3)
        print("El computador jugó: (computador)")
        # ... Lógica de Cachipún ...
        if resultado == 0:
            print("Empate")
        elif resultado == 1:
            print("Gana usuario")
        elif resultado == 2:
            print("Gana computador")
        c += 1
    print("\n... JUEGO TERMINADO ...")
    if u == 3:
        print("¡Felicidades! Ganaste la serie (u) a (c).")
    else:
        print("Perdiste. El computador ganó la serie (c) a (u).")

```

Figura 2: Evidencia del Laboratorio 10: Ejecución del sistema de votaciones y motor lógico del juego de Cachipún.

3. Principales Dificultades

A lo largo del desarrollo de la unidad, los desafíos no fueron de naturaleza algorítmica ni matemática, sino que derivaron de las limitaciones arquitectónicas del propio lenguaje Python y de las restricciones del curso:

1. **Parseo de Entradas (I/O Streams):** La captura de múltiples datos por consola resultó ser un proceso rígido. Al solicitar el ingreso de 15 calificaciones (Lab 09), la función `input()` obligaba a capturar cada valor línea por línea dentro de un ciclo. Si bien en Python es posible ingresar datos en bloque separados por espacios (mediante `.split()` y mapeo funcional), aplicar esto excedía la materia vista en clases. Esta restricción me obligó a implementar una lectura secuencial ineficiente, contrastando negativamente con la elegancia del flujo de entrada de C++ (`std::cin`), el cual procesa todo el *buffer* de datos de forma continua ignorando delimitadores automáticamente.
2. **Manejo Implícito de la Memoria:** Acostumbrado al control directo de punteros, la abstracción de memoria en Python generó una fricción técnica recurrente. En Python, el operador de asignación (`a = b`) sobre listas no genera una copia real de los datos (*deep copy*), sino que transfiere una referencia de memoria. Modificar la variable secundaria altera silenciosamente los datos de origen, obligando a mantener una trazabilidad artificial sobre el flujo de los objetos para evitar corromper la data.
3. **El Entorno de Desarrollo (IDLE):** El uso de IDLE como entorno local presentó dificultades operativas. La carencia de herramientas de desarrollo modernas (como autocompletado semántico, servidores de lenguaje *LSP* para validación de sintaxis en tiempo real e indentación inteligente) ralentizó la redacción del código. La depuración de errores se volvió un proceso excesivamente manual al carecer de un *linter* que prevenga errores de espaciado o tipado previo a la ejecución.

4. Conclusiones

La unidad de programación en Python ha sido una experiencia sumamente enriquecedora para contrastar paradigmas de desarrollo computacional. Lograr estructurar validaciones geométricas, regresiones lineales y motores lógicos demostró la innegable versatilidad de Python como lenguaje de *scripting* y prototipado rápido.

Sin embargo, desde la óptica de la Ingeniería de Software y la eficiencia asintótica, la experiencia reafirma que el alto nivel de abstracción conlleva compromisos importantes. El tipado dinámico, la asignación implícita por referencias y el manejo restrictivo de las entradas por consola facilitan el aprendizaje inicial para no especialistas, pero resultan ser un obstáculo metodológico para quien acostumbra a ejercer un control absoluto sobre el ciclo de vida del procesador y el *buffer* de memoria en lenguajes compilados. A pesar de ello, reconocer en qué contextos sacrificar rendimiento por velocidad de escritura es una habilidad clave en el diseño de soluciones tecnológicas integrales.